PAO25_25_04_Python_FUnction

Carlos Alberto Pucha

1 Funciones

1.1 ¿Qué son?

- Son un mecanismo que te ofrece el lenguaje para agrupar conjuntos de instrucciones de manera que se puedan ejecutar cuando el programador considere oportuno.
- Las funciones van identificadas por un nombre.
- Se debe usar este nombre para invocar (ejecutar) el código de la función.
- Las funciones calculan un resultado (output) en base a unos parámetros de entrada (input).
- En cada invocación de la función, los parámetros de entrada pueden variar.

1.2 Ejemplo

[1]:
```
def sumar(x, y):
    suma=x+y
    return suma
print(sumar(4,5))
print(sumar(-2,0))
```

```
9
-2
```

[2]:
```
#f = c * 1.793 + 32
c = 30
print(c * 1.793 + 32) # celsius to fahrenheit
c = 26
print(c * 1.793 + 32)
```

```
85.78999999999999
78.618
```

[4]:
```
def convert_celsius_to_fahrenheit(c):
    return c * 1.791117 + 32
```

[5]:
```
degrees = 15
print(convert_celsius_to_fahrenheit(degrees))
```

```
58.866755
```

1.3 ¿Qué ventajas nos proporcionan?

- Eliminación de duplicación de código.
- Incremento de la reutilización de código.
- Manejo de complejidad: permiten descomponer grandes sistemas en piezas más pequeñas y, por tanto, más manejables y comprensibles.

1.4 Programación de funciones en Python

Las funciones se definen por medio de la palabra clave def.
Formato general:
def name(arg1, arg2, ..., argN):
statements Muy comúnmente:
def name(arg1, arg2, ..., argN): ... return value

- Las funciones son instrucciones convencionales, que se ejecutan cuando el flujo de control llega a ellas.
- La definición de la función debe ejecutarse antes de poder invocarse la función.

[8]:
```
# Primero, definición.
def crear_diccionario(keys, values):
    return dict(zip(keys, values))
```

[9]:
```
# # Después, invocación.
nombres = ['Alfred', 'James', 'Peter', 'Harvey']
edades = [87, 43, 19, 16]
usuarios = crear_diccionario(nombres, edades)
print(usuarios)
```

```
{'Alfred': 87, 'James': 43, 'Peter': 19, 'Harvey': 16}
```

- Las funciones pueden estar anidadas en sentencias condicionales o bucles.

[11]:
```
a = -2
if a > 0:
    def operacion(x, y):
        return x + y
else:
    def operacion(x, y):
        return x * y
print(operacion(3, 4))
```

```
12
```

```
Ejercicio: Sabiendo que la fórmula general de segundo grado \
(x=\frac{-b\pm \sqrt{b^{2}-4ac}}{2a}\). Crear una función donde
el usuario mediante el teclado ingrese los valores de a, b y c;
calcular las raíces de la ecuación
```

[15]:
```
import math

# Función para calcular las raíces
def calcular_raices(a, b, c):
    # Verifico que 'a' no sea cero
    if a == 0:
        print("Error: el valor de 'a' no puede ser 0 en una ecuació
        return

    discriminante = b**2 - 4*a*c

    if discriminante > 0:
        x1 = (-b + math.sqrt(discriminante)) / (2*a)
        x2 = (-b - math.sqrt(discriminante)) / (2*a)
        print("La ecuación tiene dos raíces reales:")
        print("x1 =", x1)
        print("x2 =", x2)

    elif discriminante == 0:
        x = -b / (2*a)
        print("La ecuación tiene una única raíz real:")
        print("x =", x)

    else:
        print("La ecuación no tiene raíces reales.")


# Entrada de datos por teclado
a = float(input("Ingrese el valor de a: "))
b = float(input("Ingrese el valor de b: "))
c = float(input("Ingrese el valor de c: "))

# Llamada a la función
calcular_raices(a, b, c)
```

```
Ingrese el valor de a:  77
Ingrese el valor de b:  33
Ingrese el valor de c:  11
La ecuación no tiene raíces reales.
```

https://github.com/AnbuDigital-Ec/introduccionpythonnocturno/blob/main/funciones.ipynb